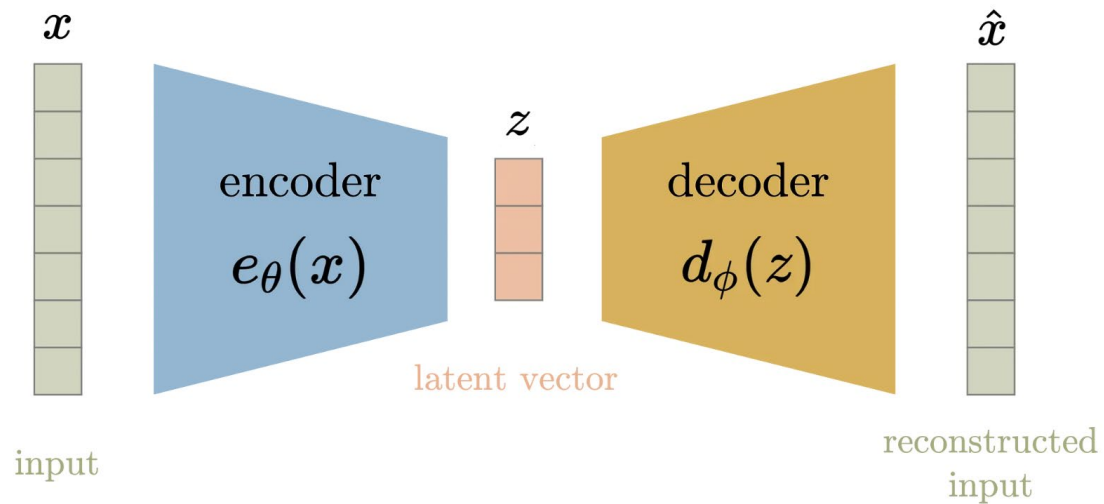


HW1: Comparison of AE and VAE

1. Show your model architectures of AE and VAE.

(1) AE Model Architecture



An autoencoder (AE) is a type of neural network used to learn effective embeddings of unlabeled data through a specific architecture. It consists of two main components: an encoder and a decoder. The encoder compresses the data from a higher-dimensional space into a lower-dimensional space, referred to as the latent space. Conversely, the decoder reverses this process, transforming the latent space back into a higher-dimensional space. The decoder's function is to ensure that the latent space retains most of the information from the original dataset by requiring it to accurately reproduce the input it receives.

The encoder comprises three convolutional layers, each utilizing ReLU activation functions. The input image, sized at (3, 50, 50), undergoes down sampling in three stages:

(3, 50, 50) $\rightarrow$ (16, 25, 25)

(16, 25, 25) $\rightarrow$ (32, 13, 13)

(32, 13, 13) $\rightarrow$ (64, 7, 7)

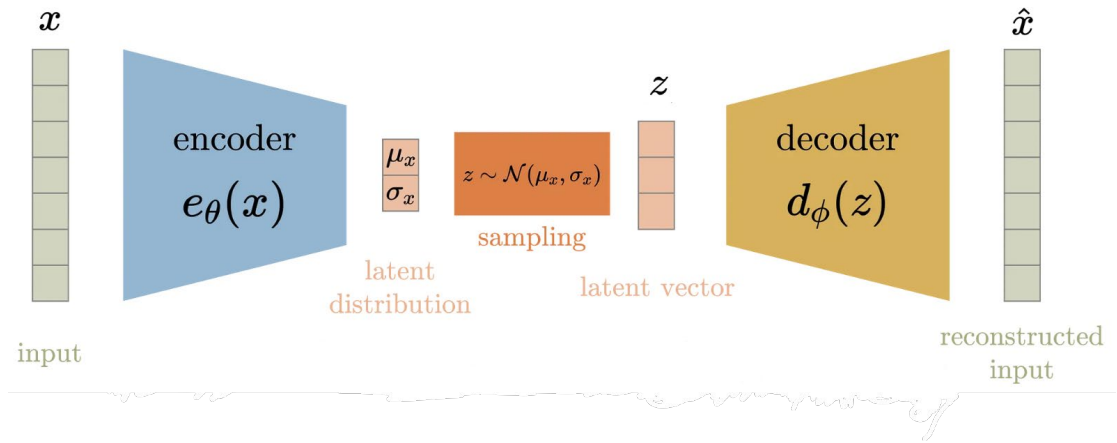
The decoder mirrors the encoder with three transposed convolutional layers, each followed by a ReLU activation, except for the last layer, which employs a Sigmoid function to scale the output back to the $[0, 1]$ range:

$(64, 7, 7) \rightarrow (32, 13, 13)$

$(32, 13, 13) \rightarrow (16, 25, 25)$

$(16, 25, 25) \rightarrow (3, 50, 50)$

(2) VAE Model Architecture



A variational autoencoder (VAE) tackles the problem of an unregularized latent space in traditional autoencoders and adds generative abilities to the entire space. Rather than producing vectors directly in the latent space, the VAE's encoder generates parameters for a specified distribution in the latent space for each input. The VAE then enforces a restriction on this latent distribution, requiring it to conform to a normal distribution. This restriction ensures that the latent space is regularized.

The encoder comprises three convolutional layers, each utilizing ReLU activation functions. The input image, sized at $(3, 50, 50)$, undergoes down sampling in three stages:

$(3, 50, 50) \rightarrow (16, 25, 25)$

$(16, 25, 25) \rightarrow (32, 13, 13)$

$(32, 13, 13) \rightarrow (64, 7, 7)$

The outputs of encoder are then flattened, and two fully connected layers produce the mean (μ) and log-variance ($\log\sigma^2$) for the latent space. The reparameterization function generates the latent vector z from μ and $\log\sigma^2$.

The **latent dimension** is set to **128** as it strikes a good balance between compressing data and accurately reconstructing images.

The decoder mirrors the encoder with three transposed convolutional layers, each followed by a ReLU activation, except for the last layer, which employs a Sigmoid function to scale the output back to the $[0, 1]$ range:

$(64, 7, 7) \rightarrow (32, 13, 13)$

$(32, 13, 13) \rightarrow (16, 25, 25)$

$(16, 25, 25) \rightarrow (3, 50, 50)$

2. Show your implementation detail, such as batch size, epoch, learning rate, data preprocessing, etc. in detail. Describe how you decide on the hyperparameters to obtain the final AE and VAE models.

(1) Data Preprocessing

I applied transformations to convert the images from NumPy arrays to PyTorch tensors and normalized them to the $[0, 1]$ range. Additionally, I transformed the image data type from float64 to float32, as this is the standard for input into neural networks.

(2) Data Loader

The data loader was configured with a **batch size** of **4**, which specifies the number of samples processed by the network during each iteration. Additionally, I enabled `shuffle=True` to randomize the order of the data, which aids in improving the model's generalization. To ensure reproducibility, I set a seed, guaranteeing that the program's results remain consistent across different runs—an essential aspect for experiments and debugging.

(3) Training Configuration

Loss function for AE:

I used the **Mean Squared Error (MSE)**, which measures the difference between the input image and its reconstruction.

Loss function for VAE:

Binary Cross-Entropy Loss (BCE) is employed, with the reduction method set to 'sum' to suit the pixel-wise characteristics of image data. Besides, **KL divergence** is incorporated to regularize the latent space, with a **weight of 0.1** to maintain a balance between reconstruction and regularization.

Hyperparameters of both AE and VAE:

I selected the **Adam optimizer** due to its ability to effectively adapt the learning rate, which can lead to faster convergence. At the same time, multiple tests indicate that setting the **learning rate** to **0.001** yields the best training results.

I monitored the training loss to choose **80** as the number of **epochs** because it represented the optimal balance between underfitting and overfitting.

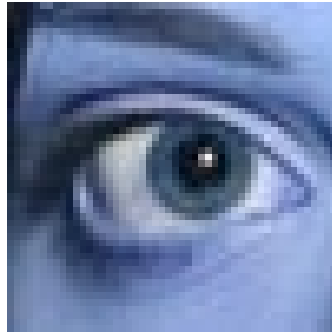
3. Show the quantitative and qualitative results of experiments for both AE and VAE models in Step 4, Step 5, and Step 6.

(1) Quantitative Results

Model	Average PSNR	Average SSIM
AE	34.39	0.98
VAE	27.38	0.90
AE with Gaussian noise = 0.5	32.57	0.97
VAE with Gaussian noise = 0.5	24.99	0.85

(2) Qualitative Results

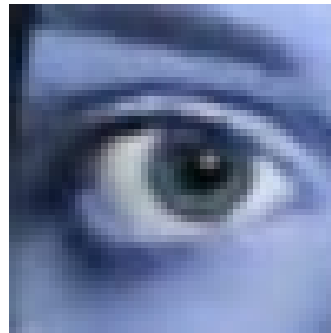
Original



AE



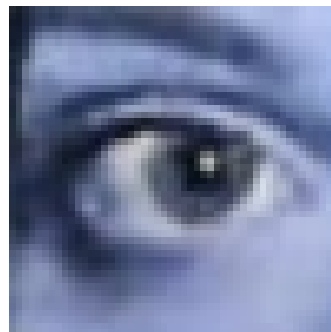
VAE



**AE
with Gaussian noise**



**VAE
with Gaussian noise**



4. Compare the differences between AE and VAE, and explain the reasons for them. You may use quantitative and qualitative results you have obtained to evidence your thought.

(1) Quantitative Analysis:

Comparison of AE and VAE

The AE model demonstrates excellent average PSNR and SSIM scores, signifying that it generates high-quality reconstructions with minimal distortion. The elevated SSIM score indicates a strong structural similarity between the original and reconstructed images, suggesting that the AE effectively captures intricate details. In contrast, the VAE model, while still performing adequately, has lower PSNR and SSIM scores than the AE model. The reduced PSNR implies that the VAE introduces more distortion in the reconstruction process, likely due to the regularization from the KL divergence term, which aims to create a meaningful latent space. Although the SSIM score remains relatively high, it indicates that some structural details may be compromised during reconstruction compared to the AE.

Comparison of AE and VAE with same Gaussian noise

When Gaussian noise is introduced into the latent vector of the AE model, there is a slight decline in both PSNR and SSIM. The drop in PSNR suggests that the noise slightly impacts the quality of the reconstruction, although the model continues to perform adequately. The minimal decrease in SSIM indicates that the AE remains robust to structural changes despite the added noise. In contrast, the VAE model experiences a more pronounced reduction in both PSNR and SSIM with the introduction of Gaussian noise, indicating that it is more susceptible to such disturbances. This increased sensitivity may be attributed to the regularization effect of the latent space, which makes the model more vulnerable to input perturbations. The lower PSNR reflects more noticeable reconstruction errors, while the decline in SSIM signifies a reduction in structural similarity.

Summary:

AE vs. VAE

The AE model consistently achieves better reconstruction quality than the VAE model because the AE is not limited by the requirement to maintain a structured latent space, a constraint that the VAE must adhere to.

Effect of Gaussian noise

Both models exhibit decreased performance when Gaussian noise is introduced, with the VAE experiencing a more significant impact. This indicates that although both models can handle some level of noise, the AE demonstrates greater resilience, while the VAE's latent space regularization makes it more sensitive to noise-related degradation.

(2) Qualitative Analysis:

Comparison of AE and VAE

The AE reconstruction closely mirrors the original image, with well-preserved details around the eye, including the iris and eyelid. The colors look natural, and the overall sharpness is retained. It is clear that the AE model has effectively captured the key features of the original image with minimal detail loss or color distortion, as indicated by the high PSNR and SSIM values, demonstrating its effectiveness in producing accurate reconstructions. In contrast, the VAE reconstruction exhibits some loss of detail compared to the original image. The edges are somewhat blurred, and the overall sharpness is diminished. Moreover, there is a slight shift in color tones, resulting in a less vibrant appearance than the original. This suggests that the VAE model introduces some blurring and detail loss, which aligns with the lower PSNR and SSIM values. This outcome is anticipated due to the regularization effect of the KL divergence in the VAE, which sacrifices reconstruction quality for a more organized latent space.

Comparison of AE and VAE with same Gaussian noise

The AE reconstruction with Gaussian noise preserves much of the original image's detail, though there is a slight increase in noise, especially in smoother regions. The colors remain true to the original, and the eye's structure is well-maintained. While the addition of Gaussian noise has somewhat affected the AE's ability to create a perfectly clear reconstruction, the overall quality is still high. This is evident in the slight decline in PSNR and SSIM values, which demonstrate the model's resilience to noise. In contrast, the VAE reconstruction with Gaussian noise shows more noticeable blurring and detail loss compared to the original image. The edges are less sharp, and the colors seem a bit faded. The Gaussian noise intensifies the VAE's tendency to produce blurred images, resulting in a more significant loss of detail and lower structural similarity. This

is reflected in the further decrease in PSNR and SSIM values, underscoring the VAE's vulnerability to noise.

Summary:

AE vs. VAE

The AE model reliably generates more precise and intricate reconstructions compared to the VAE, regardless of the presence of Gaussian noise. The VAE compromises some level of reconstruction quality in exchange for a more organized latent space, leading to images that appear somewhat blurred and lack detail.

Effect of Gaussian noise

While both models experience some degradation in reconstruction quality with the addition of Gaussian noise, the AE model is more resilient, maintaining better detail and color accuracy. The VAE model, on the other hand, shows a more noticeable decline in quality, with increased blurring and color distortion.

(3) Conclusion:

Overall, the AE model excels in generating high-quality reconstructions, whereas the VAE model offers a more organized latent space, albeit with a slight reduction in reconstruction accuracy. Introducing Gaussian noise emphasizes the differences in robustness between the two models.

5. What can we do to preserve the high-frequency information in the original images in your AE or VAE models? Please discuss this problem and do some experiments to prove it.

High-frequency information in images often corresponds to details such as edges, textures, and fine structures. I incorporated two different loss functions into the evaluation process: total variation loss and frequency-domain loss. Finally, I analyzed the quantitative and qualitative results of these methods.

I conducted experiments using VAE to investigate this issue.

(1) Quantitative Results

Additional Loss Function	Average PSNR	Average SSIM
Total Variation Loss with $\lambda = 0.1$	26.94	0.87
Frequency-Domain Loss with $\lambda = 0.5$	27.30	0.90

Quantitative Analysis:

Comparison of Total Variation Loss and Frequency-Domain Loss

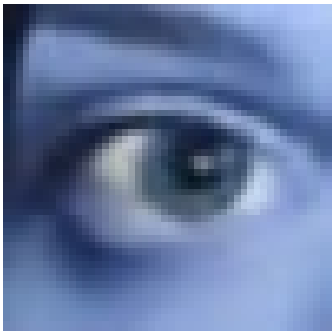
Frequency-Domain Loss (with $\lambda = 0.5$) outperforms Total Variation Loss (with $\lambda = 0.1$), indicating that it maintains higher fidelity to the original image’s pixel values and structural details. Frequency-Domain Loss provides a balance between noise reduction and structure preservation, leading to slightly sharper and clearer results. Total Variation Loss tends to smooth out fine details, which may be desirable in noise reduction contexts but at the cost of losing important image details and textures.

(2) Qualitative Results

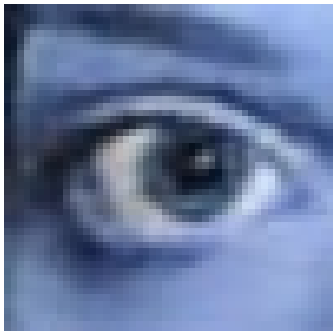
Original



Total Variation Loss



Frequency-Domain Loss



Qualitative Analysis:

Performance of Total Variation Loss

The image reconstructed with Total Variation Loss appears more blurred. The edges are less defined, and there's a noticeable reduction in sharpness, particularly around the eye and eyebrow areas. The overall image looks softer and less detailed. Conclusion, Total Variation Loss prioritizes smoothness and removes noise, but at the cost of fine details and sharpness, resulting in a more overly smoothed appearance.

Performance of Frequency-Domain Loss

The reconstruction using Frequency-Domain Loss closely resembles the original. The details around the eye are well-preserved, with clear edges and good color fidelity. There is a slight softening, but the overall structure and features remain intact. Conclusion, Frequency-Domain Loss effectively captures high-level features and textures, making it well-suited for maintaining a realistic and detailed appearance in the reconstructed image.

6. If we use a very deep Neural Network with a large number of neurons to construct AE and VAE, will the evaluation results increase? Why or why not? Please conduct experiments to demonstrate it.

Deeper AE Model Architecture

```
AutoEncoder(  
  (encoder): Sequential(  
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (1): ReLU()  
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (3): ReLU()  
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (5): ReLU()  
    (6): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (7): ReLU()  
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (9): ReLU()  
  )  
  (decoder): Sequential(  
    (0): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1))  
    (1): ReLU()  
    (2): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (3): ReLU()  
    (4): ConvTranspose2d(64, 32, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (5): ReLU()  
    (6): ConvTranspose2d(32, 16, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (7): ReLU()  
    (8): ConvTranspose2d(16, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1))  
    (9): Sigmoid()  
  )  
)
```

Deeper VAE Model Architecture

```
VariationalAutoEncoder(  
  (encoder): Sequential(  
    (0): Conv2d(3, 16, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (1): ReLU()  
    (2): Conv2d(16, 32, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (3): ReLU()  
    (4): Conv2d(32, 64, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (5): ReLU()  
    (6): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (7): ReLU()  
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1))  
    (9): ReLU()  
  )  
  (fc_mu): Linear(in_features=1024, out_features=128, bias=True)  
  (fc_logvar): Linear(in_features=1024, out_features=128, bias=True)  
  (fc_decode): Linear(in_features=128, out_features=1024, bias=True)  
  (decoder): Sequential(  
    (0): ConvTranspose2d(256, 128, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1))  
    (1): ReLU()  
    (2): ConvTranspose2d(128, 64, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (3): ReLU()  
    (4): ConvTranspose2d(64, 32, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (5): ReLU()  
    (6): ConvTranspose2d(32, 16, kernel_size=(4, 4), stride=(2, 2), padding=(2, 2), output_padding=(1, 1))  
    (7): ReLU()  
    (8): ConvTranspose2d(16, 3, kernel_size=(4, 4), stride=(2, 2), padding=(1, 1))  
    (9): Sigmoid()  
  )  
)
```

(1) Quantitative Results

Model	Average PSNR	Average SSIM
AE	34.39	0.98
VAE	27.38	0.90
Deeper AE	27.38	0.89
Deeper VAE	24.46	0.80

Quantitative Analysis:

Both the average PSNR and SSIM of deeper AEs and VAEs showed significant degradation, suggesting that increasing model complexity does not always lead to improved performance.

(2) Qualitative Results

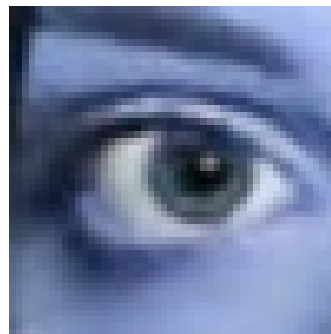
Original



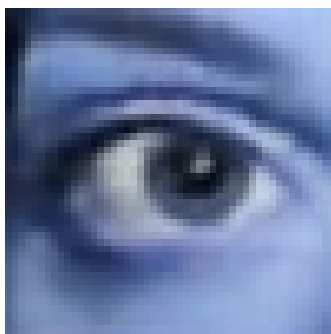
AE



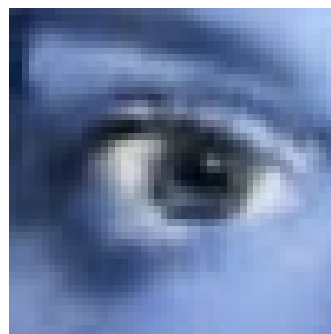
VAE



Deeper AE



Deeper VAE



Qualitative Analysis:

Performance of Deeper AE

The deeper AE model produces an image with noticeable blurring and a slight loss of detail compared to the standard AE. While the general structure of the eye is preserved, the finer details and clarity are compromised.

Performance of Deeper VAE

The Deeper VAE shows the most significant degradation, with substantial blurring and a notable loss of fine details. The image appears smudged, with poor edge definition and distorted colors, making it the least accurate reconstruction.

(3) Conclusion

Impact of Depth on Performance:

Adding more layers generally increases the representational capacity of the model, allowing it to capture more complex patterns in the data. However, this also increases the risk of overfitting, especially if the dataset is small. Additionally, deeper networks require more computational resources and can be more challenging to train effectively due to issues like vanishing/exploding gradients.

References

[Variational AutoEncoders \(VAE\) with PyTorch - Alexander Van de Kleut \(avandekleut.github.io\)](https://avandekleut.github.io)

[Variational Autoencoder \(VAE\)—PyTorch Tutorial | by Reza Kalantar | Nov, 2022 | Medium | Medium](#)

[Modern PyTorch Techniques for VAEs: A Comprehensive Tutorial | Hunter Heidenreich, Machine Learning Engineer](#)

[Difference between AutoEncoder \(AE\) and Variational AutoEncoder \(VAE\) | by Aqeel Anwar | Towards Data Science](#)

[Autoencoder vs Variational Autoencoder \(VAE\): Differences, Example \(vitalflux.com\)](https://vitalflux.com)

[Pytorch 手把手實作-AutoEncoder. 這邊文章的架構為 | by Tommy Huang |](#)

[Medium](#)

[CNN 經典論文實戰\(四\)--Autoencoder - iT 邦幫忙::一起幫忙解決難題，拯救 IT 人的一天 \(ithome.com.tw\)](#)

[Implement Convolutional Autoencoder in PyTorch with CUDA - GeeksforGeeks](#)

[GitHub - LukeDitria/CNN-VAE: Variational Autoencoder \(VAE\) with perception loss implementation in pytorch](#)

[【CNN 基础】常见的 loss 函数及其实现\(一\)——TV Loss tvloss-CSDN 博客](#)

[GitHub - EndlessSora/focal-frequency-loss: \[ICCV 2021\] Focal Frequency Loss for Image Reconstruction and Synthesis](#)